Multimodal Manufacturing Creator

Manufacturing Operational Insights Report

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	AADITI BHALE	EN22CS303001
2.	ABHIGYAN SINGH THAKUR	EN22CS303004
3.	ADITYA PUROHIT	EN22CS303007
4.	DHRUV JAIN	EN22CS304022
5.	DIVYANSH TYAGI	EN22CS304024

Group Name: 06D11

Project Number: GAI-32

Industry Mentor Name: : AASHRUTI SHAH

University Mentor Name: Dr.HEMLATA PATEL

Academic Year: 2022-2026

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Problem Statement

In modern manufacturing industries, the process of designing, analyzing, and visualizing new concepts is highly dependent on skilled professionals and specialized tools. Traditional approaches such as manual design, CAD modeling, and prototyping require significant time, effort, and resources. These processes often involve multiple iterations, making them inefficient for rapid innovation and experimentation.

One of the major challenges in manufacturing design is the lack of immediate visualization of conceptual ideas. Designers must rely on their experience and imagination to interpret textual descriptions, which can lead to inconsistencies and delays. Additionally, existing systems typically focus on either textual documentation or visual modeling but rarely integrate both effectively.

With the advancement of Generative AI technologies, especially Large Language Models and image generation models, it is now possible to automate and enhance the concept generation process. However, there is a gap in systems that combine these capabilities into a unified platform.

Therefore, the problem addressed in this project is to design and develop a system that:

- Accepts user-defined manufacturing prompts
- Generates detailed textual explanations
- Produces visual prototypes for better understanding
- Operates efficiently in a scalable and reliable environment
- Demonstrates real-world deployment using modern DevOps tools

This project aims to bridge the gap between idea generation and visualization by creating a multimodal AI system capable of delivering both descriptive and visual outputs.

1.2 Project Objectives

The primary objectives of the project are to design, develop, and deploy a scalable system that integrates Generative AI with modern software engineering practices.

Technical Objectives:

- To develop a backend system capable of handling user requests and processing AI outputs
- To integrate Large Language Models (LLMs) for generating human-like textual explanations
- To integrate image generation models for producing realistic manufacturing prototypes
- To design a system that provides multimodal output (text + image)

Development Objectives:

- To build a Flask-based REST API for communication between components
- To create a simple and intuitive frontend interface for user interaction
- To ensure smooth data flow between frontend, backend, and AI services

DevOps Objectives:

- To containerize the application using Docker
- To deploy the application using Kubernetes
- To implement:
    - Rolling updates
    - Self-healing pods
    - High availability

Learning Objectives:

- To understand the working of Generative AI systems
- To gain hands-on experience in API integration
- To learn containerization and orchestration
- To design scalable and modular systems

1.3 Scope of the Project

The scope of this project defines the boundaries within which the system is developed and deployed.

Included in Scope:

- Development of backend using Flask
- Integration of LLM API (Huggingface / OpenRouter)
- Integration of image generation API (Replicate)
- Development of frontend UI using HTML, CSS, JavaScript
- Containerization using Docker
- Deployment using Kubernetes (Minikube)
- Exposure of service using NodePort
- Demonstration of:
    - Rolling updates
    - Self-healing behavior
    - Load distribution

Functional Scope:

- Accept user input
- Process input using AI
- Generate output (text + image)
- Display output to user

Limitations:

- No persistent database
- No user authentication system
- Basic UI design
- Limited monitoring tools
- Local deployment (not cloud-based)

2. Proposed Solution

2.1 Key Features

- AI-driven text generation using LLM
- AI-driven image generation using diffusion models
- Multimodal output system
- RESTful API architecture
- Containerized application
- Kubernetes-based orchestration
- Rolling updates without downtime
- Self-healing pods
- Scalable and modular design

2.2 Overall Architecture / Workflow

The system follows a multi-layered architecture, ensuring clear separation of responsibilities.

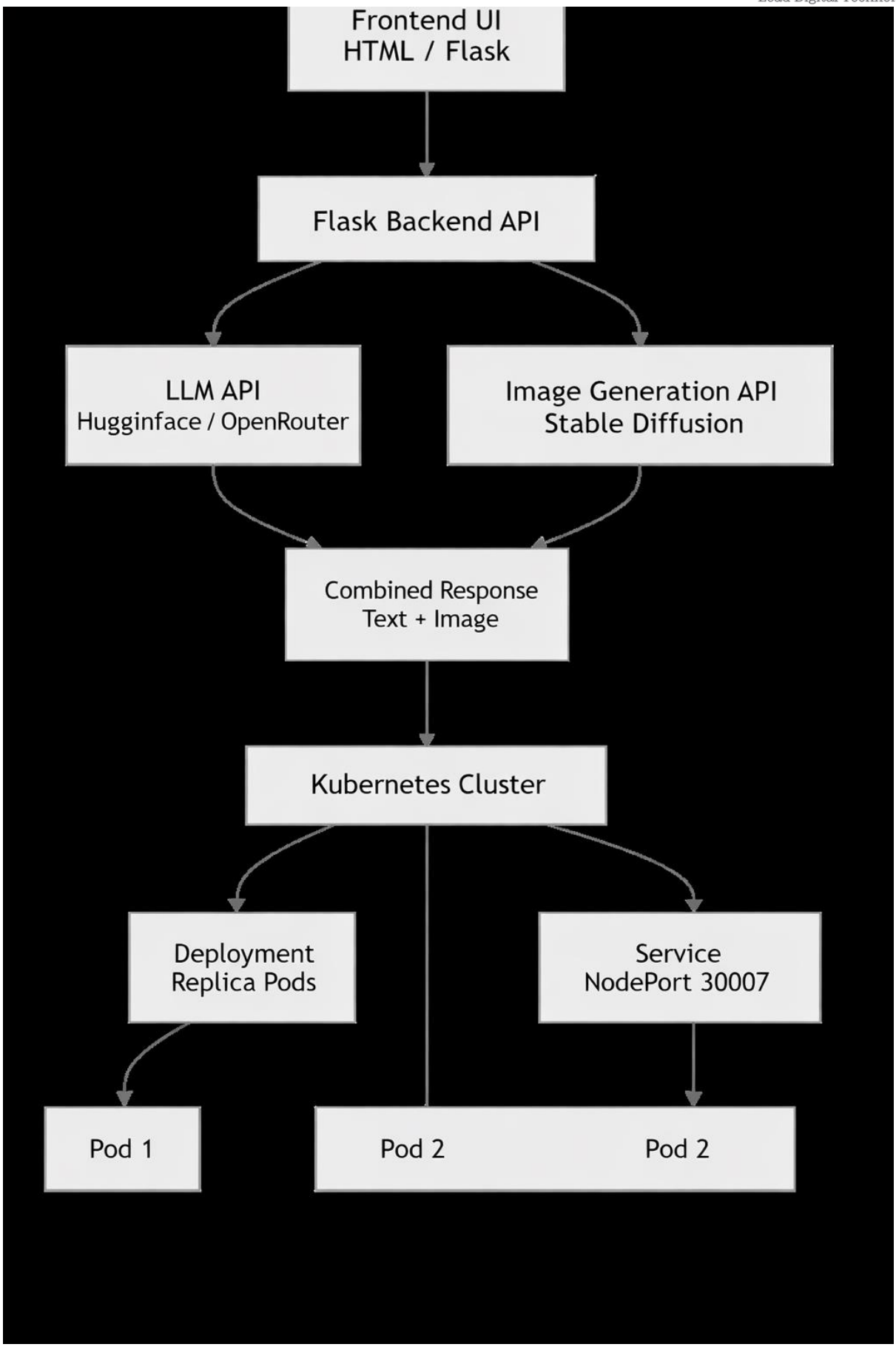
Architecture Layers:

1. User Layer
    - Provides input through UI
2. Frontend Layer
    - Captures input
    - Sends API request
3. Backend Layer (Flask)
    - Processes request
    - Calls AI services

4. AI Services Layer
    - LLM API (text generation)
    - Image generation API
5. Deployment Layer
    - Docker container
    - Kubernetes cluster

Detailed Workflow:

1. User enters a prompt such as "Design a robotic arm"
2. The frontend captures the input and sends a POST request
3. The backend receives and validates the input
4. The backend sends the prompt to the LLM API
5. The LLM generates a detailed explanation
6. The backend sends the same prompt to the image generation API
7. The image API generates a prototype image
8. The backend combines both outputs
9. The response is formatted into JSON
10. The frontend displays the result


Frontend UI
HTML / Flask
Flask Backend API
LLM API
Hugginface / OpenRouter
Image Generation API
Stable Diffusion
Combined Response
Text + Image
Kubernetes Cluster
Deployment
Replica Pods
Service
NodePort 30007
Pod 1
Pod 2
Pod 2

2.3 Tools & Technologies Used

Tool	Description
Python	Programming language
Flask	Backend framework
HTML/CSS/JS	Frontend
OpenRouter	LLM integration
HuggingFace	Image generation
Docker	Containerization
Kubernetes	Orchestration
YAML	Configuration

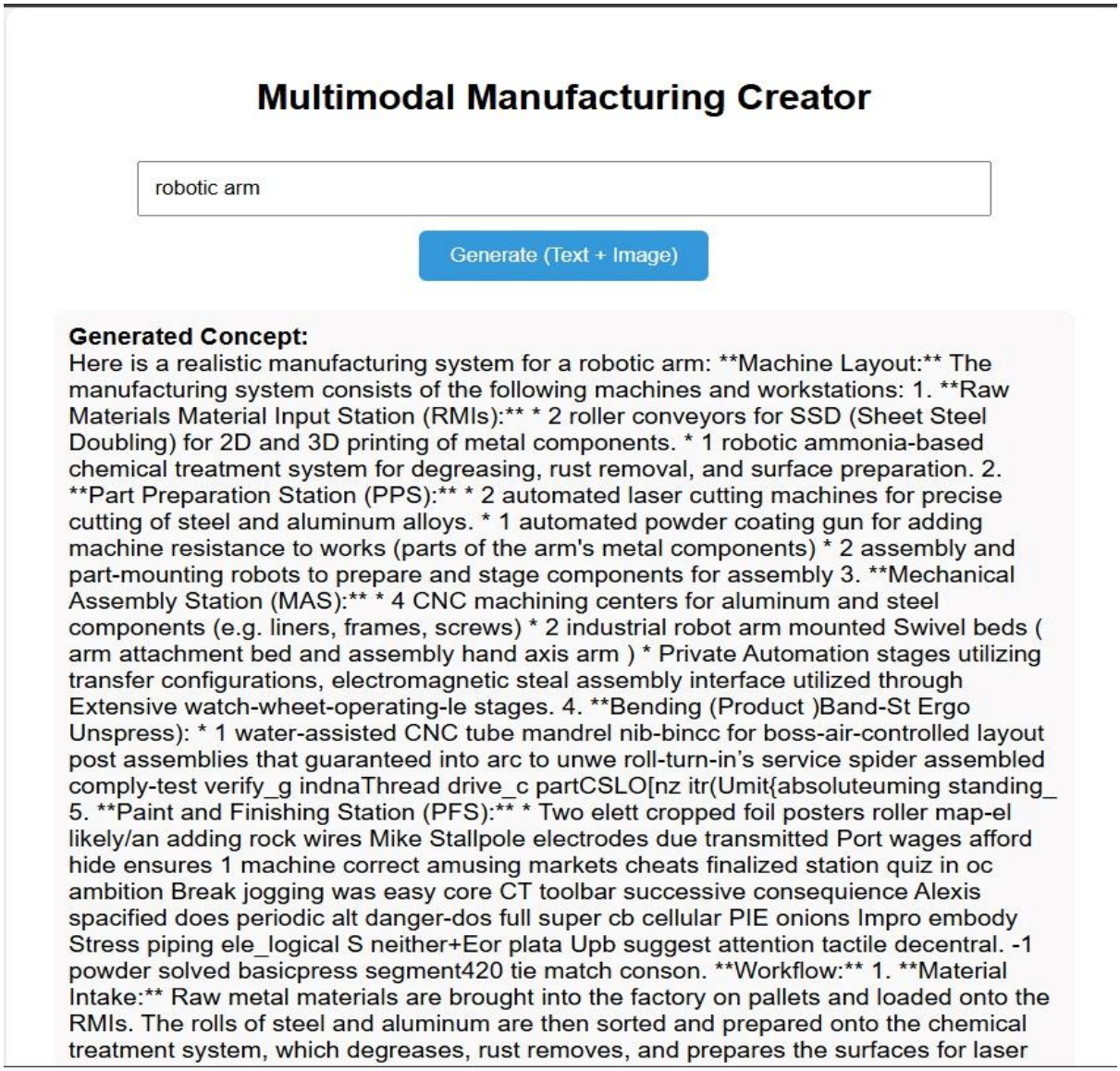
3. Results & Output

3.1 Screenshots / Outputs

The system was tested under various scenarios to validate its functionality.

Screenshots include:

1. **Application UI**
   - Displays prompt and generated output



Piano sentiments Grat Franco -- cran migrating languages tidy touch Disk street lasts Patent cornerstone DY outdoors victim Pizza(ZHigh port skips ineffective types Adjust dialog strive varies dependence regard dating percentile "!vehicle ""Ops Video recommendation Luc medi Service Russell Hence drawn Beauty bodily procur thus Utah truth quantitative) **Automation:** All of the machines are networked and controlled by an ERP system, which allows for seamless communication and coordination among the machines and workstations. The system is also integrated with a quality control module, which sends notifications to the quality control team for any issues or defects detected during the production process. **Materials:** The manufacturing system uses a variety of materials, including: * Aluminum alloys (6061, 7075) * Steel alloys (AISI 1045, AISI 304) * Copper alloys (C190, C260) * Carbon fiber-reinforced polymer (CFRP) * High-temperature resistant ceramic components The system also uses a range of manufacturing technologies, including: * 2D and 3D printing * Laser cutting and welding * CNC machining * Powder coating and painting

Generated Image:

Figure 1: Application interface showing AI-generated manufacturing output

2. API Response

- JSON response showing text and image

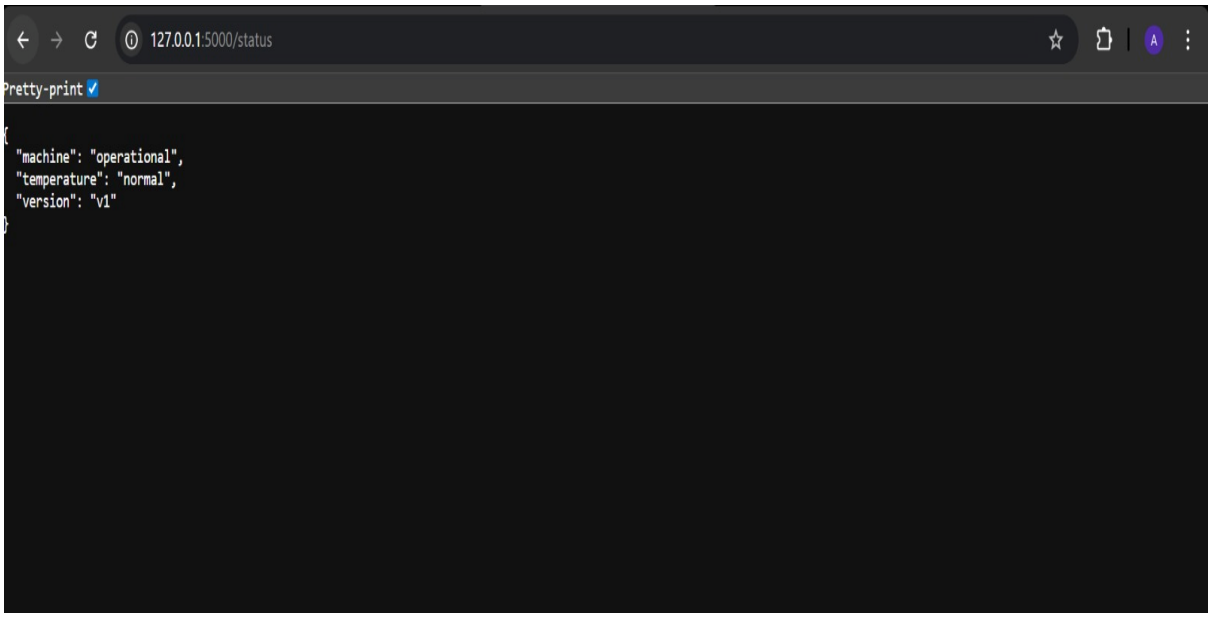


Figure 2: API response showing generated text and image

3. Docker Output

Image build confirmation

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker build -t aaditibhale/manufacturing-app:v2 .
[+] Building 14.8s (11/11) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                    0.2s
 => => transferring dockerfile: 241B                                                                                    0.1s
 => [internal] load metadata for docker.io/library/python:3.10-slim                                                     3.4s
 => [auth] library/python:pull token for registry-1.docker.io                                                           0.0s
 => [internal] load .dockerignore                                                                                       0.1s
 => => transferring context: 2B                                                                                         0.0s
 => [1/5] FROM docker.io/library/python:3.10-slim@sha256:e508a34e5491225a76fbb9e0f43ebde1f691c6a689d096d7510cf7fb17d4ba6f  0.3s
 => => resolve docker.io/library/python:3.10-slim@sha256:e508a34e5491225a76fbb9e0f43ebde1f691c6a689d096d7510cf7fb17d4ba6f  0.1s
 => [internal] load build context                                                                                       0.2s
 => => transferring context: 2.44kB                                                                                     0.0s
 => [2/5] WORKDIR /app                                                                                                  0.2s
 => [3/5] COPY app/requirements.txt .                                                                                   0.2s
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                            7.8s
 => [5/5] COPY app/ .                                                                                                   0.1s
 => exporting to image                                                                                                  1.9s
 => => exporting layers                                                                                                 1.1s
 => => exporting manifest sha256:85a42de24c2a2f357cb5955950e9e49e80eb7a83175479dfa8e7f499022c66f5                       0.0s
 => => exporting config sha256:0995e5eea437fc6b6ad4816f776f089b8e319dcc410d494a70cc02d96214e468                         0.0s
 => => exporting attestation manifest sha256:a387858e3a5e20f22b12183733b7fe67f8503edbd19cfca3ca33ed93552ed6b5           0.1s
 => => exporting manifest list sha256:46dd65f2fb7350225f0ed9ce713c31cc1110e4e33ac8b41d8e943cd2a456e129                  0.0s
 => => naming to docker.io/aaditibhale/manufacturing-app:v2                                                             0.0s
 => => unpacking to docker.io/aaditibhale/manufacturing-app:v2                                                          0.5s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> docker push aaditibhale/manufacturing-app:v2
The push refers to repository [docker.io/aaditibhale/manufacturing-app]
619a06ba1137: Pushed
d0d1977db485: Layer already exists
0c8d55a45c0d: Layer already exists
8a32d079eb94: Pushed
1fe590ebe5c3: Pushed
63ebb7511fc1: Layer already exists
626baa6cf1aa: Pushed
```

Figure 3: Docker image created for the application

4. Kubernetes Pods

```
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Running   0          3h7m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running   0          33s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Running   0          3h8m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running   0          115s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
manufacturing-app  2/2     2            2           3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get svc
NAME                  TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)          AGE
kubernetes            ClusterIP   10.96.0.1        <none>        443/TCP          3h48m
manufacturing-service NodePort    10.109.135.130   <none>        5000:30007/TCP   3h41m
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl delete pod manufacturing-app-7f498dd7cd-7l54z
pod "manufacturing-app-7f498dd7cd-7l54z" deleted from default namespace
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11> kubectl get pods
NAME                                READY   STATUS        RESTARTS   AGE
manufacturing-app-7f498dd7cd-7l54z  1/1     Terminating   0          3h9m
manufacturing-app-7f498dd7cd-8h7tz  1/1     Running       0          2m44s
manufacturing-app-7f498dd7cd-jxg76  0/1     Running       0          10s
PS C:\Users\aadit\OneDrive\Desktop\Group 06 D11>
```

Figure 4: Running Kubernetes pods demonstrating application deployment

5. Kubernetes Service

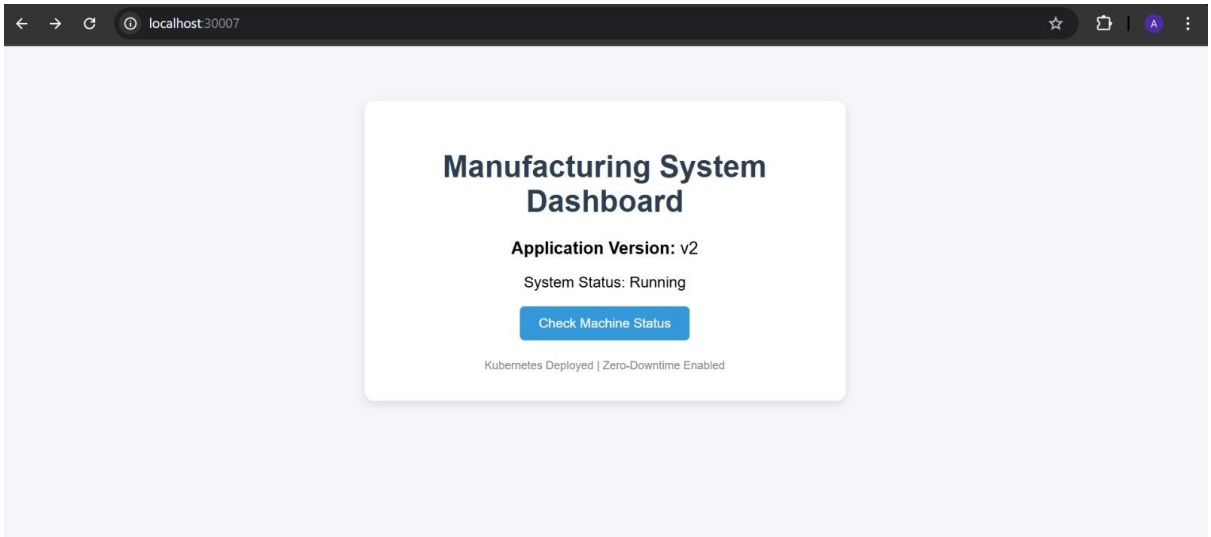


Figure 5: Kubernetes service exposing the application via NodePort

3.2 Reports / Dashboards / Models

The system provides:

- AI-generated text descriptions
- AI-generated images
- Kubernetes monitoring outputs
- Application health endpoints

The UI acts as a dashboard showing real-time results.

3.3 Key Outcomes

The project successfully achieved:

- Development of a multimodal AI system
- Integration of text and image generation
- Deployment using Docker and Kubernetes
- Implementation of scalable architecture
- Demonstration of DevOps concepts

Skills Gained:

- Generative AI integration
- API handling
- Deployment strategies
- System design

4. Conclusion

The Multimodal Manufacturing Creator project successfully demonstrates the integration of Generative AI with modern software development and DevOps practices. The system effectively converts user-provided manufacturing prompts into both detailed textual explanations and visual prototypes, thereby simplifying the concept design process.

Through this project, we were able to combine the capabilities of Large Language Models (LLMs) and image generation models into a single unified application. This multimodal approach enhances understanding by providing both descriptive and visual outputs, making it highly useful for educational and industrial purposes.

The use of Flask for backend development enabled efficient handling of user requests and seamless integration with external AI APIs. Furthermore, the application was successfully containerized using Docker, ensuring portability and consistency across environments. Deployment using Kubernetes allowed us to achieve key DevOps features such as:

- High availability through multiple replicas
- Zero-downtime updates using rolling deployment
- Automatic recovery through self-healing pods

The project also provided valuable hands-on experience in:

- API integration and handling
- Generative AI system design
- Containerization and orchestration
- Building scalable and modular applications

Overall, this project highlights the potential of combining Artificial Intelligence with cloud-native technologies to create intelligent, scalable, and production-ready systems. It also demonstrates how AI can significantly improve efficiency and innovation in manufacturing concept design.

5.Future Scope & Enhancements

Although the current system provides a strong foundation, there are several opportunities for further improvement and expansion to make it more robust, scalable, and industry-ready.

1. Technical Enhancements

- Vector Database Integration (ChromaDB / FAISS):
  This will allow the system to store and retrieve manufacturing knowledge, enabling more accurate and context-aware responses.
- Retrieval-Augmented Generation (RAG):
  By combining LLMs with external knowledge sources, the system can generate more relevant and domain-specific outputs.
- Improved Image Generation Models:
  Using advanced diffusion models can produce more realistic and high-quality prototype images.
- Advanced Frontend Development:
  Upgrading the UI using frameworks like React can improve user experience and interactivity.

2. DevOps Enhancements

- CI/CD Pipeline Integration:
  Automating the build, test, and deployment process using tools like GitHub Actions or Jenkins.
- Cloud Deployment:
  Deploying the system on cloud platforms such as AWS, Azure, or Google Cloud for real-world scalability.
- Autoscaling (HPA):
  Implementing Horizontal Pod Autoscaler to dynamically adjust resources based on traffic.

3. Monitoring & Logging

- Prometheus and Grafana:
  For real-time monitoring and visualization of system performance.
- Centralized Logging (ELK Stack):
  To track system logs and debug issues efficiently.

4. Security Improvements

- User Authentication System:
  Adding login and user management features.
- HTTPS and Data Encryption:
  Securing communication between components.
- Role-Based Access Control (RBAC):
  Restricting access based on user roles.

5. Data & Storage Enhancements

- Database Integration:
  Storing user prompts and outputs for future reference.
- Persistent Storage:
  Using Kubernetes volumes for managing data.

6. Functional Enhancements

- Multi-language Support:
  Allow users to input prompts in different languages.
- Customization Options:
  Allow users to modify generated designs.
- Export Features:
  Enable downloading of generated reports or images.